Data Structure

The Introduction to Data Structures

- What is Data?
- The difference between Data and Information.
- What is Data Structure?
  - Organize data in efficiently
- Real-life examples of Data Structures

Data Types vs. Abstract Data Types

- Data types: defines values and operation
- Primitive and user defined datatype
- Difference between data type and abstract data type
- The use of abstract data types
- -2^31 to +(2^31-1)

Data Structure Definition & Advantages

- Definition of Data Structures
- Which Data Structure to choose
- Advantages of Data Structures
  - Efficiency, reusability, abstraction
- Linear and non-linear data structures
- Static and dynamic data structures

Basics of Asymptotic Analysis

- Definition of data structures.
- How to measure the efficiency of data structure
- Example - Comparing two data structures.
- Discussion on the need to study the time complexity

Basics of Asymptotic Analysis

- How to calculate the time complexity?
- Method 1: Calculation of time complexity using the exact running time (not practical)

Basics of Asymptotic Analysis

- Finding the approximate time complexity.
- Calculating the growth rate of time complexity function.
- Asymptotic complexity
  - Approximate measure of time complexity

$f(n) = 5n^2 + 6n + 12$

n	$5n^2$	6n	12
1	21.74%	26.09%	52.17%
10	87.41%	10.49%	2.09%
100	98.79%	1.19%	0.02%
1000	99.88%	0.12%	0.0002%

It is a clear observation that for larger values of n, the squared term consumes almost 99 % of time.

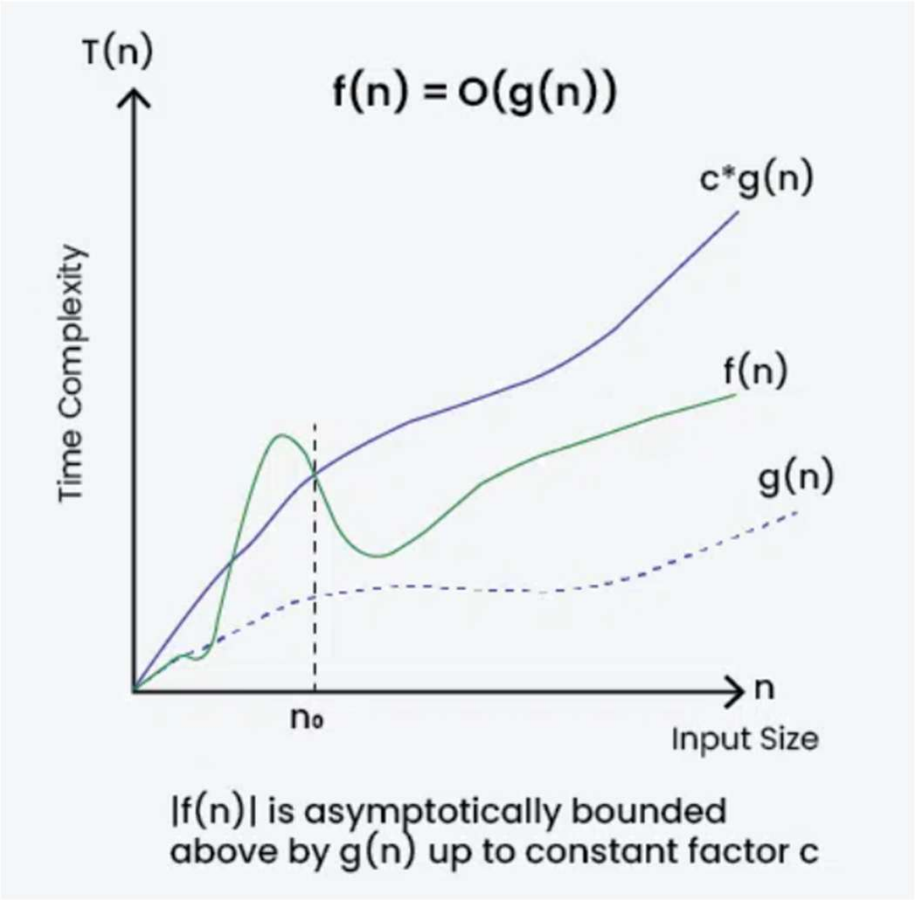
Basics of Asymptotic Analysis

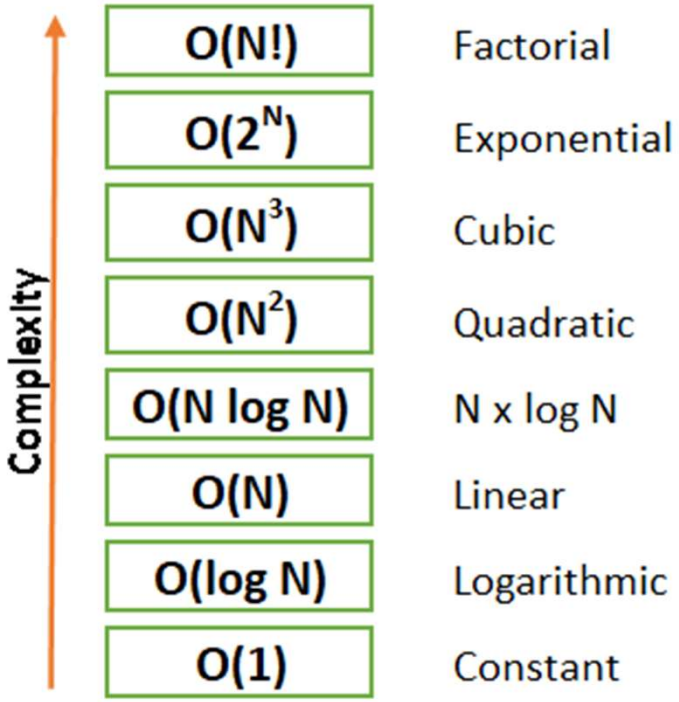
- Definition of Big O notation
  - used to measures performance of algorithm by providing the order of growth of function
  - Gives least upper bound or worst-case on time complexity function f(n)
- Big O notation
  - If f(n) and g(n) are two functions, then

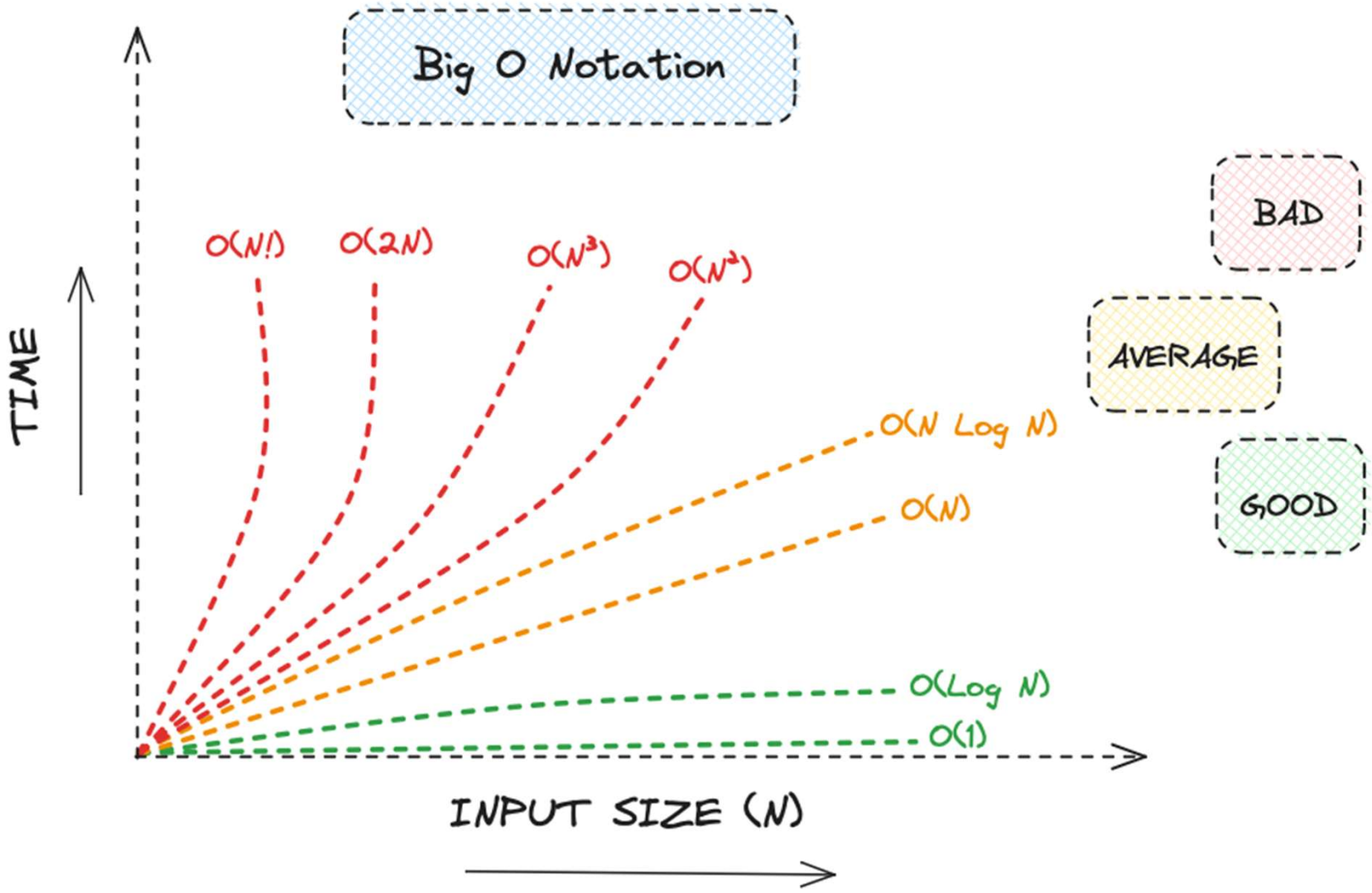
    $f(n) = O(g(n))$
  - If there exits a constant c and n0 such that:

    $f(n) \leq c.g(n)$, $\quad$ for all $n \geq n0$

  This simply means that f(n) does not grow faster than g(n)


T(n)
f(n) = O(g(n))
c*g(n)
f(n)
g(n)
Time Complexity
n
n₀
Input Size
|f(n)| is asymptotically bounded
above by g(n) up to constant factor c


Complexity
O(N!)
Factorial
O(2^N)
Exponential
O(N^3)
Cubic
O(N^2)
Quadratic
O(N log N)
N x log N
O(N)
Linear
O(log N)
Logarithmic
O(1)
Constant


Big O Notation
BAD
AVERAGE
GOOD
O(N!)
O(2N)
O(N³)
O(N²)
O(N Log N)
O(N)
O(Log N)
O(1)
TIME
INPUT SIZE (N)

Basics of Asymptotic Analysis

- Programming example demonstrating the importance of Big O notation in determining the growth rate of a function

Calculate growth rate of function or worst case time complexity

```
int main()
{
    int i, sum = 0, n;          <-- 1 time
    scanf("%d", &n);            <-- 1 time

    for(i=1; i<=n; i++) {
        sum = sum + i;          <-- n times
    }

    printf("%d", sum);          <-- 1 time
    return 0;                   <-- 1 time
}
```

f(n) = O(n), linear growth

Guidelines for Asymptotic Analysis

- Calculation of Time Complexity when we have: a) Loops b) Nested loops c) Consecutive statements
- Calculation of Time Complexity when we have an if-then-else statement
- Logarithmic Complexity

- for(i=1; i<=n; i=i*2)

  printf(‘hello’);

  - N K
  - 1 1
  - 2 2
  - 4 3
  - 8 4
  - N = 2^(k-1) -> log(N) = (k-1) -> k = log(N)-1

What is the time complexity of the following program:

```
void fun(int n) {
    int i, j;
    for(i=1; i<=n/3; i++)
        for(j=1; j<=n; j += 4)
            printf("Hello\n");
}
```

```
void fun(int n) {
    int i, j;
    for(i=1; i<=n/3; i++)
        for(j=1; j<=n; j += 4)
            printf("Hello\n");
}
```

$$n = 1 + (k - 1) * 4$$

$$n - 1 = (k - 1) * 4$$

$$\frac{n - 1}{4} + 1 = k$$

Iter 1: j = 1

Iter 2: j = 1 + 4

Iter 3: j = 1 + 4 + 4 = 1 + 2*4

Iter 4: j = 1 + 3*4

⋮

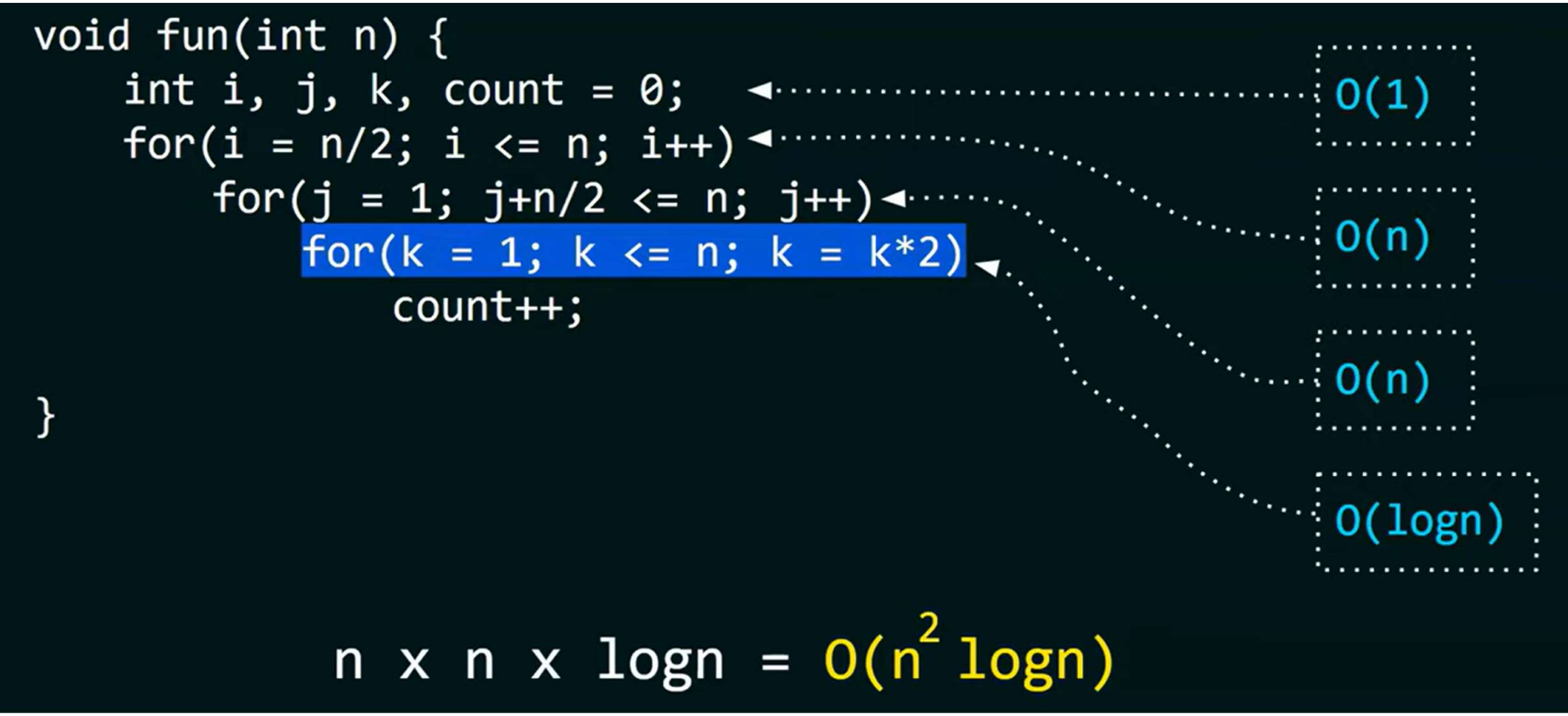
Iter k: j = n = 1 + (k-1)*4

TIME COMPLEXITY: O(N)

What is the time complexity of the following program?

```
void fun(int n) {
    int i, j, k, count = 0;
    for(i = n/2; i <= n; i++)
        for(j = 1; j+n/2 <= n; j++)
            for(k = 1; k <= n; k = k*2)
                count++;

}
```


void fun(int n) {
int i, j, k, count = 0;
for(i = n/2; i <= n; i++)
for(j = 1; j+n/2 <= n; j++)
for(k = 1; k <= n; k = k*2)
count++;
}
O(1)
O(n)
O(n)
O(logn)
n x n x logn = O(n^2 logn)

What is the time complexity of the following program:

```
void fun(int n) {
    if(n <= 1) return;
    int i, j;
    for(i=1; i<=n; i++)
        for(j=1; j<=n; j++) {
            printf("Hello\n");
            break;
        }
}
```

What is the time complexity of the following program:

```
void fun(int n) {
    int i = 1;
    while(i<n) {
        int j = n;
        while(j>0)
            j = j/2;
        i = 2*i;
    }
}
```